

Integrating your Chatbot into a Web Interface



Introduction

In this lab, you will learn to set up a back-end server and integrate your chatbot into a web application.

Learning objectives

After completing this lab, you will be able to:

- Set up your back-end server
- Integrate your chatbot into your Flask server
- Communicate with the back-end using a web page

Prerequisites

This section assumes you know how to build the simple terminal chatbot explained in the first lab.

There are two things you must build to create your ChatGPT-like website:

1. A back-end server that hosts your chatbot
2. A front-end webpage that communicates with your back-end server

Without further ado, let's get started!

Step 1: Hosting your chatbot on a backend server

What is a backend server?

A backend server is like the brain behind a website or application. In this case, the backend server will receive prompts from your website, feed them into your chatbot, and return the output of the chatbot back to the website, which will be read by the user.

Hosting a simple backend server using Flask

Note: Consider using a requirements.txt file

flask is a Python framework for building web applications with Python. It provides a set of tools and functionalities to handle incoming requests, process data, and generate responses, making it easy to power your website or application.

Prerequisites

For all terminal interactions in this lab (such as running python files or installing packages), you will use the built-in terminal that comes with Cloud IDE. You may launch the terminal by either:

- Pressing **Ctrl + `**, or
- By selecting Terminal --> New Terminal from the toolbar at the top of the IDE window on the right.

In your terminal, let's install the following requisites:



```
python3.11 -m pip install flask
python3.11 -m pip install flask_cors
```

Setting up the server

Next, you will create a script that stores your flask server code.

To create a new Python file, Click on **File Explorer**, then right-click in the explorer area and select **New File**. Name this new file **app.py**.

Let's take a look at how to implement a simple flask server:

```
<> py 
```

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

Paste the above code in the `app.py` file you just created and save it.

In this code:

- You import the `Flask` class from the `flask` module.
- You create an instance of the `Flask` class and assign it to the variable `app`.
- You define a route for the homepage by decorating the `home()` function with the `@app.route()` decorator. The function returns the string 'Hello, World!'. This means that when the user visits the URL where the website is hosted, the backend server will receive the request and return 'Hello, World!' to the user.
- The `if __name__ == '__main__':` condition ensures that the server is only run if the script is executed directly, not when imported as a module.
- Finally, you call `app.run()` to start the server.

```
<> bash 
```

```
python3.11 app.py
```

 Run

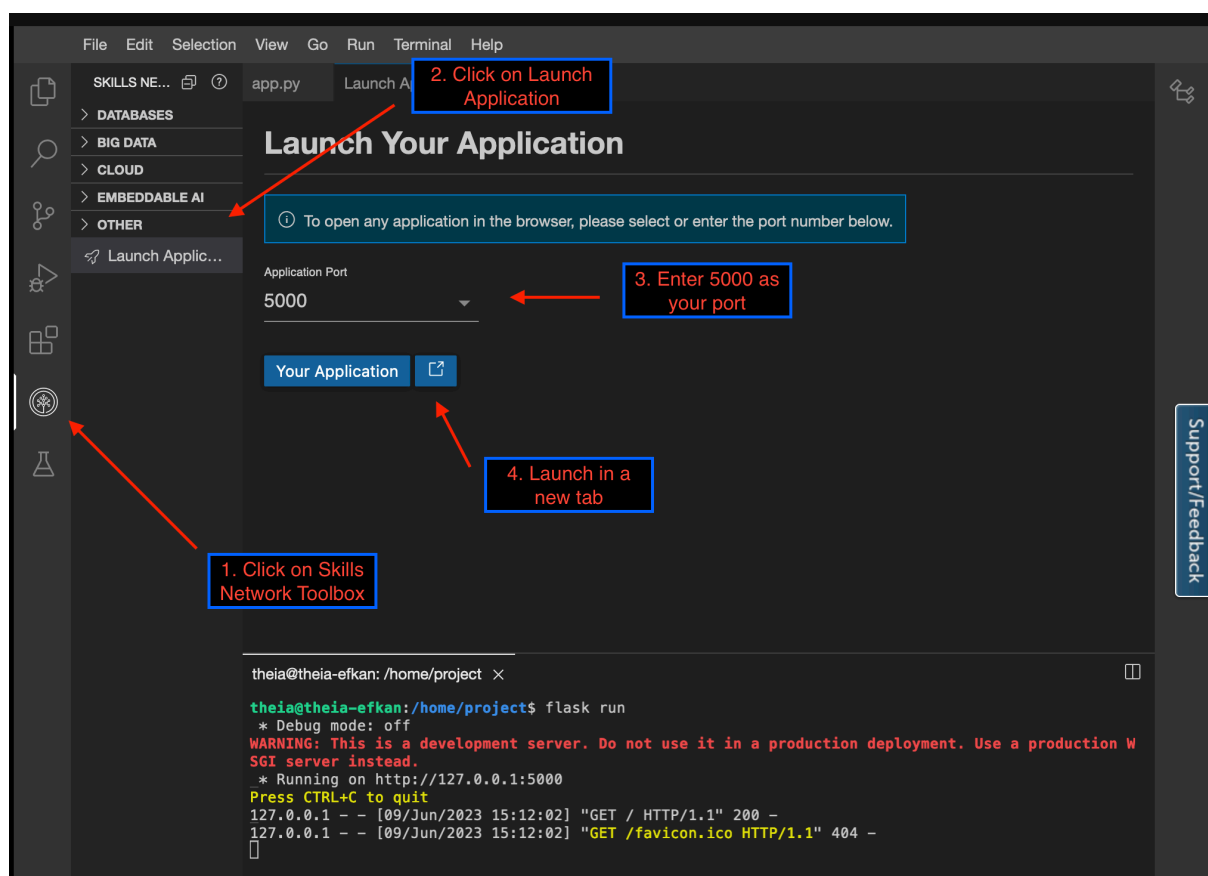
Save this code in a Python file, for example, `app.py`, and run it by typing `python app.py` in the terminal. By default, Flask hosts the server at <http://127.0.0.1:5000/> (which is equivalent to <http://127.0.0.1:5000/>).

With this command, the Flask server will start running. If you run this server on your

<http://localhost:5000/> in your web browser.

However, you are currently running this lab in the Skills Network Cloud. Thus, you can access your server as follows:

1. Navigate to the Skills Network Toolbox from the toolbar on the left side of the IDE
2. Click "Launch Application" in the adjacent vertical sidebar
3. Enter 5000 as your Application Port
4. Launch the application in a new browser tab

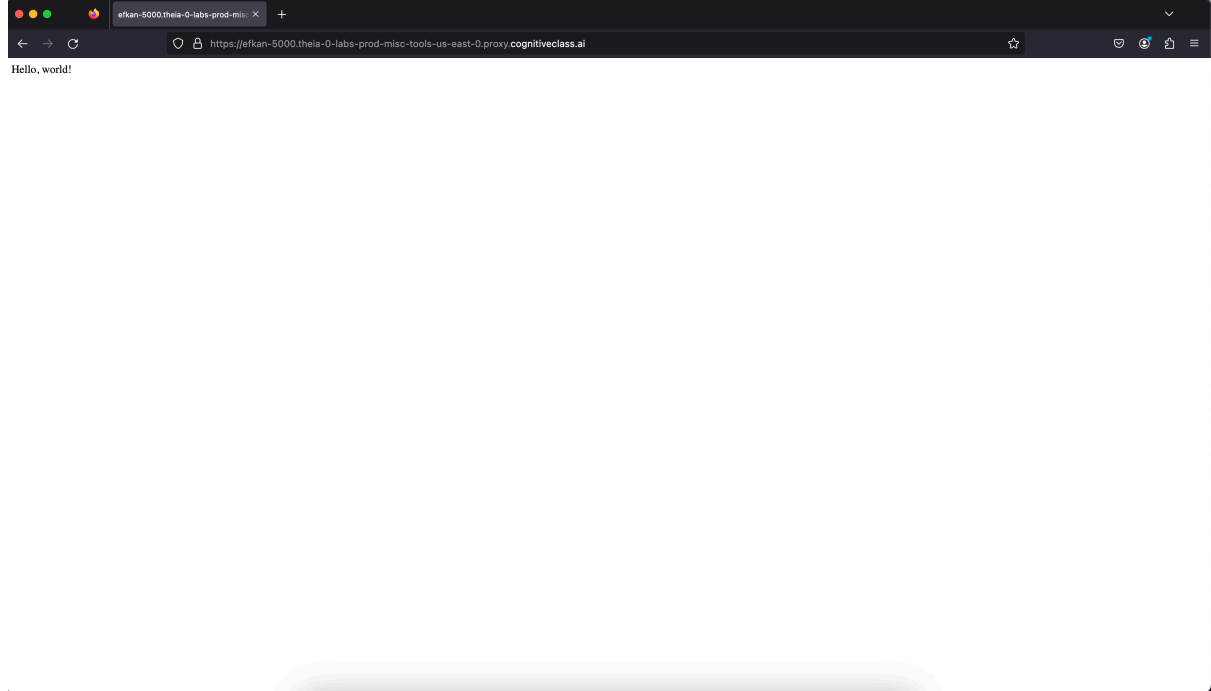


By performing the above steps, you have visited the **relative localhost URL** of the cloud server.

IMPORTANT: Throughout the rest of this lab, you will refer to this URL as .

On visiting the localhost, you should see the "Hello, World!" message displayed.

Here's what it should look like:

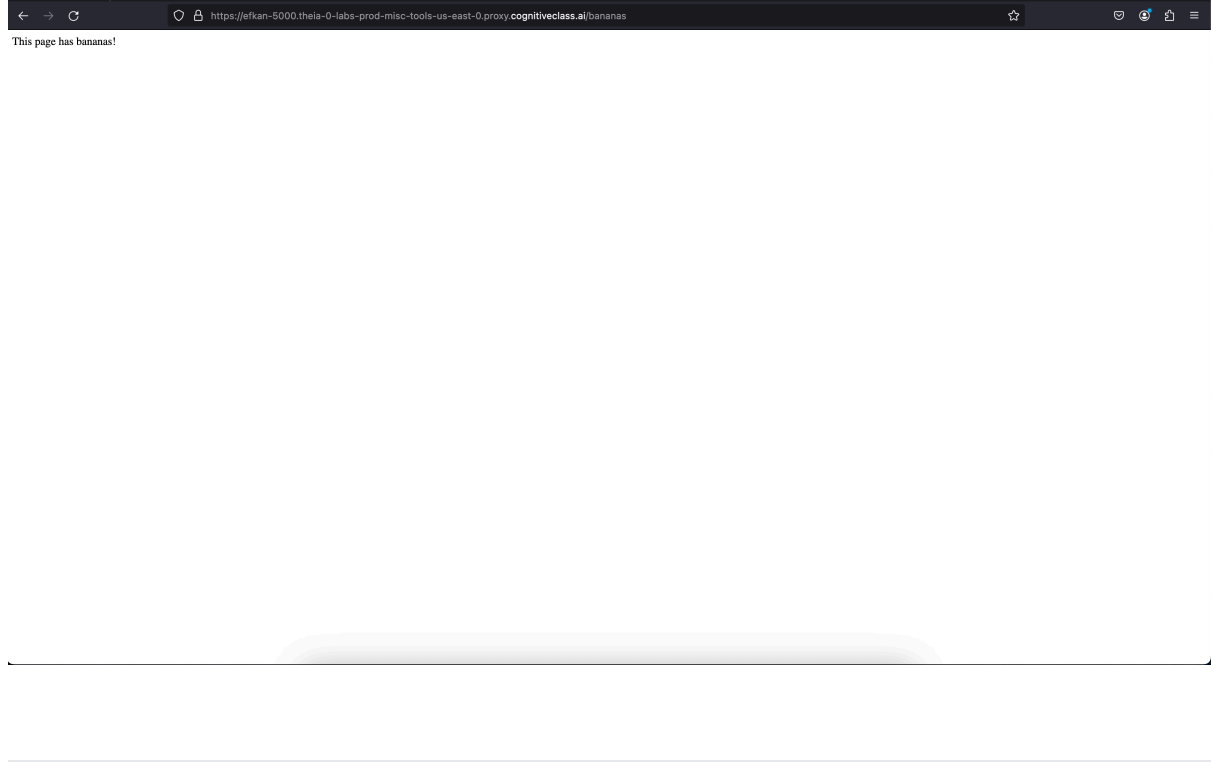


Let's add the following routes to try it out:

```
@app.route('/bananas')
def bananas():
    return 'This page has bananas!'

@app.route('/bread')
def bread():
    return 'This page has bread!'
```

Now, let's stop our app using **Ctrl + C** in the terminal and re-run with **flask run**. Then, let's visit both of these routes at /bananas and /bread. Here's what you should see:



Okay - now that you've demonstrated how routes work, you can remove these two routes (bananas and bread) from your `app.py` as you won't be using them.

Before proceeding, you'll also add two more lines of code to your program to mitigate CORS errors - a type of error related to making requests to domains other than the one that hosts this webpage. You'll be modifying your code as follows:



py



```
from flask import Flask
from flask_cors import CORS      # newly added

app = Flask(__name__)
CORS(app)                        # newly added

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

Integrating your chatbot into your Flask server

Now that you have your Flask server set up, let's integrate your chatbot into your Flask server.

As stated at the beginning, this lab assumes you've completed the first lab of this guided project on how to create your own simple chatbot.

First, you'll install the requisites We pin library versions to ensure the code runs consistently without breaking due to future updates or changes in dependencies.



plaintext



```
python3.11 -m pip install transformers==4.41.2
python3.11 -m pip install torch==2.2.2
python3.11 -m pip install accelerate==0.30.1
python3.11 -m pip install numpy==1.26.4
```

Next, let's copy the code to initialize your chatbot from lab 1 and place it at the top of your script. You also must import the necessary libraries for your chatbot.



py



```
from transformers import AutoModelForSeq2SeqLM
from transformers import AutoTokenizer
```



py



```
model_name = "facebook/blenderbot-400M-distill"  
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)  
tokenizer = AutoTokenizer.from_pretrained(model_name)  
conversation_history = []
```

Next, you'll need to import a module to read the data.



py



```
from flask import request
```

Before implementing the actual function though, you need to determine the structure you expect to receive in the incoming HTTP request.

Let's define your expected structure as follows:



json



```
{  
  "prompt": "message"  
}
```

Now implement your chatbot function. Again, you'll copy code over from your chatbot implementation from the first lab.



py



```
@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    data = request.get_json()
    input_text = data["prompt"]

    # Keep only recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate response
    outputs = model.generate(
        *inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    # Decode response
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Save conversation
    conversation_history.append(f"User: {input_text}")
    conversation_history.append(f"Bot: {response}")

    return response
```

The only new thing you've done up till now is read the prompt from the HTTP request body. You've copied everything else from your previous chatbot implementation!



py



```
from flask import Flask, request, render_template
from flask_cors import CORS
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

app = Flask(__name__)
CORS(app)

model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []

@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    # Read prompt from HTTP request body
    data = request.get_json()
    input_text = data["prompt"]

    # Keep only the recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize the input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate the response
    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    # Decode the response
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)

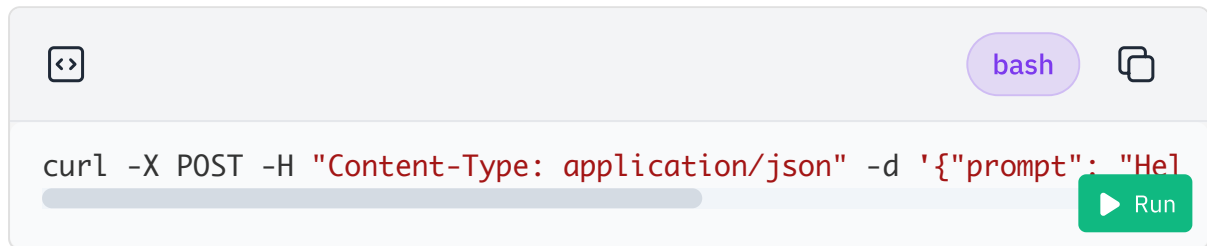
    # Add interaction to conversation history
```

```
    return response

if __name__ == '__main__':
    app.run()
```

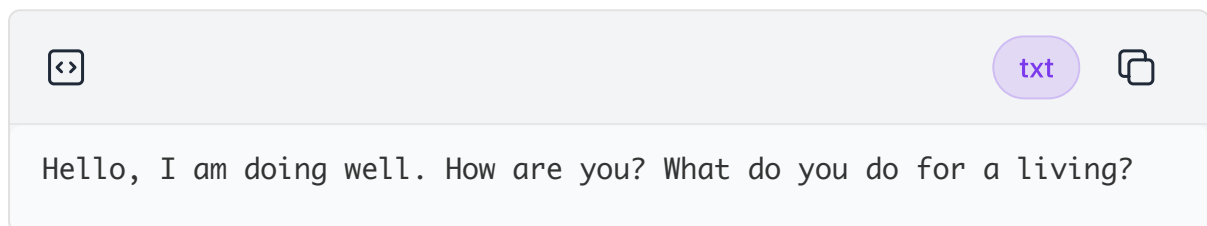
Now let's test your implementation by using `curl` to make a POST request to /chatbot with the following request body: {'prompt': 'Hello, how are you today?'}.

Open a new terminal: Select terminal tab --> open new terminal



A terminal window with a light gray header bar containing a code icon on the left, a 'bash' tab in the center, and a copy icon on the right. The main area shows a red `curl` command: `curl -X POST -H "Content-Type: application/json" -d '{"prompt": "Hello, how are you today?"}'`. A green 'Run' button with a play icon is at the bottom right.

Here's the output of the above code:



A terminal window with a light gray header bar containing a code icon on the left, a 'txt' tab in the center, and a copy icon on the right. The main area shows the chatbot's response: `Hello, I am doing well. How are you? What do you do for a living?`

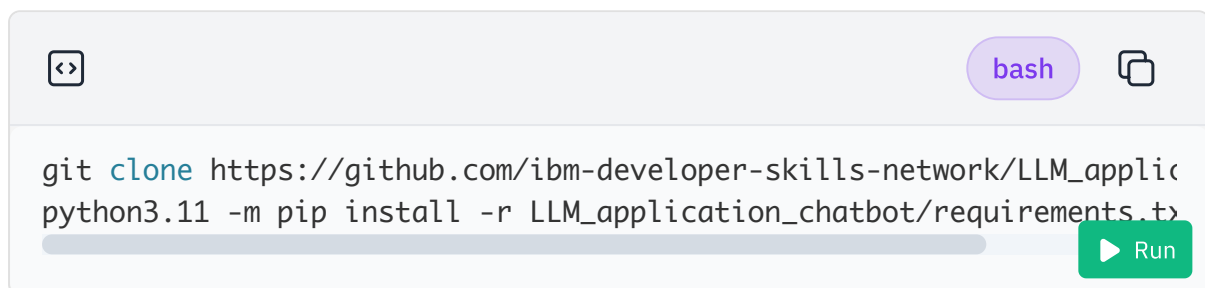
If you got a similar response, then congratulations! You have successfully created a Flask backend server with an integrated chatbot!

After you finish, press `Ctrl + c` to stop the server.

Communicating with your backend using a webpage

In this section, you'll download a template chatbot webpage and configure it to make requests to your backend server.

First, let's clone a repository that has a template website and install your required libraries. If your flask app is running, terminate it with **Ctrl + C** and run the following lines in the terminal:



A terminal window with a light gray background. The title bar shows a code icon, a 'bash' button, and a copy icon. The terminal contains the following text: `git clone https://github.com/ibm-developer-skills-network/LLM_application_chatbot/` followed by `python3.11 -m pip install -r LLM_application_chatbot/requirements.txt`. A green 'Run' button is visible at the bottom right of the terminal area.

If the operations are complete with no errors, then you have successfully obtained a copy of the template repository.

The file structure of this repo should be as follows:

- LLM_application_chatbot/
 - static/
 - script.js
 - < other assets >
 - templates/
 - index.html

Let's move your flask app **app.py** to the **LLM_application_chatbot/** folder so that you can host **index.html** on your server.

Both **app.py** and the **LLM_application_chatbot/** folder should be in **/home/project**. You can move **app.py** into **LLM_application_chatbot/** by running the following line in the terminal:

<>

bash

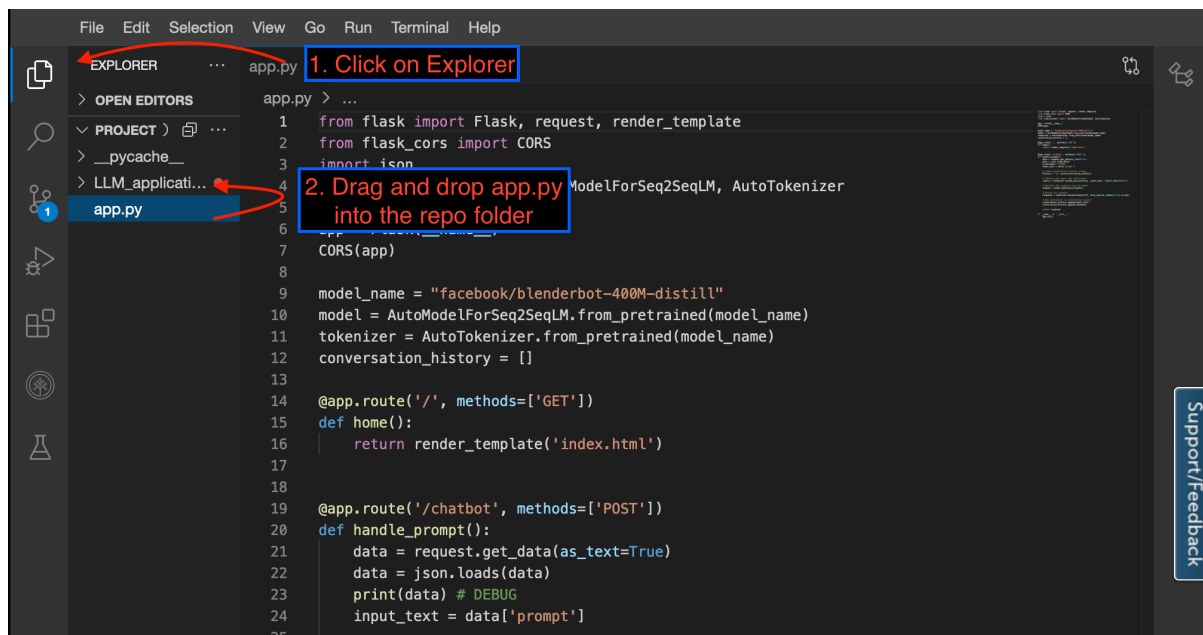
📄

mv app.py LLM_application_chatbot/

▶ Run

Alternatively, you may do the same by dragging and dropping in the IDE as follows:

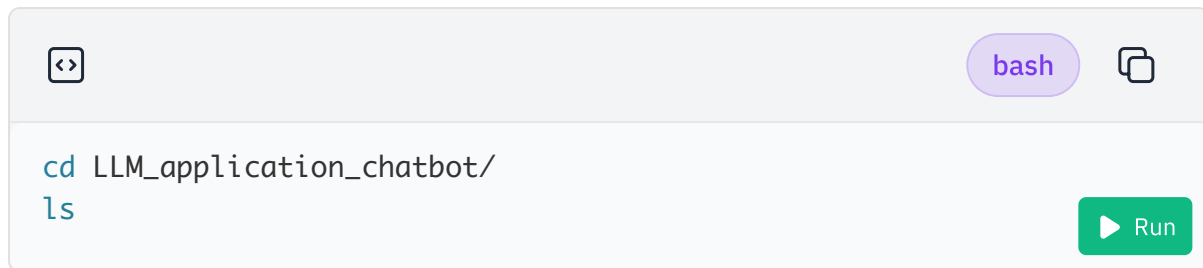
1. Navigate to the Explorer from the sidebar on the left
2. Drag and drop `app.py` into `LLM_application_chatbot/` (Be careful not to drop it in a subfolder)



After adding your flask app, the file structure should be as follows:

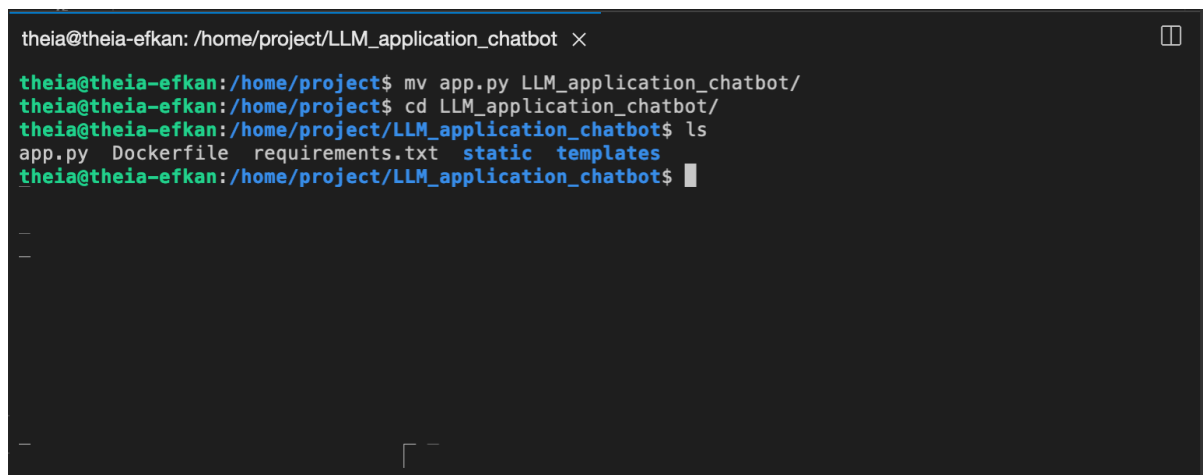
- LLM_application_chatbot/
 - *app.py*
 - static/
 - script.js
 - < other assets >
 - templates/

Let's test this by running:



A terminal window with a light gray header bar containing a code icon, a 'bash' button, and a copy icon. The main area has a dark background with the following text: `cd LLM_application_chatbot/` and `ls`. A green 'Run' button with a play icon is in the bottom right corner.

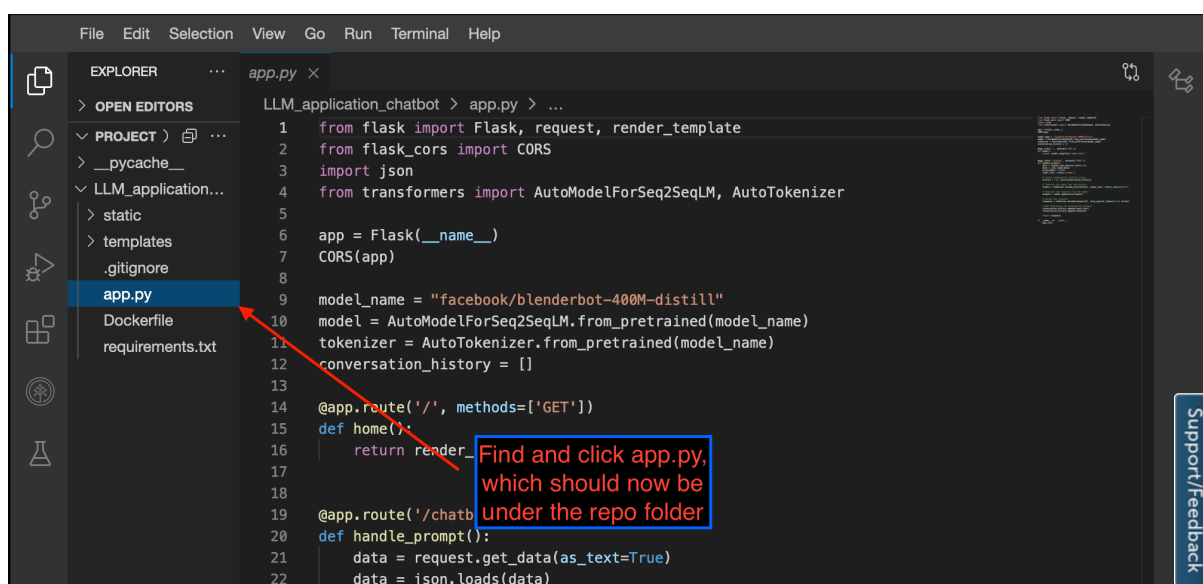
You should see `app.py` in the output:



A terminal window with a dark background. The prompt is `theia@theia-efkan: /home/project/LLM_application_chatbot`. The user enters `mv app.py LLM_application_chatbot/`, then `cd LLM_application_chatbot/`, and finally `ls`. The output shows `app.py Dockerfile requirements.txt static templates`.

Since you've changed the location of `app.py`, you must re-open it in your editor. This is because the current tab for `app.py` tries to read from and write to `app.py` at a path that no longer exists.

Simply close the editor tab for `app.py`, and re-open it by clicking `app.py` in the explorer.



Now, let's modify your `app.py` so that you host `index.html` at `/`. You can achieve this by adding the following route to your code:

```
@app.route('/', methods=['GET'])
def home():
    return render_template('index.html')
```

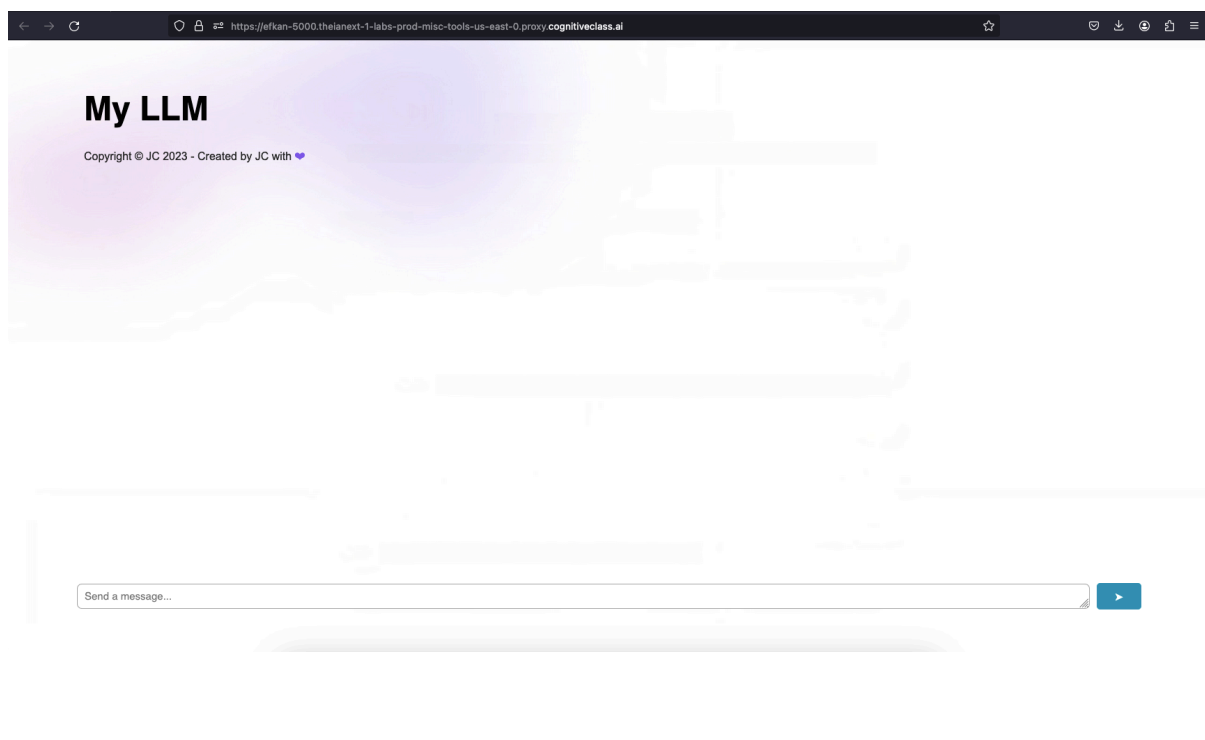
After adding the code, you can run your flask app with the following:

```
flask run
```

Run

Once your flask server is running, you may see the render of `index.html` by visiting `/`.

Here's what you should see:



This template already has JavaScript code that emulates a chatbot interface. When you type a message and hit send, the website template does the following:

1. Enter your input message into a text bubble
2. Send your input to an endpoint (by default, this is set to www.example.com in the template)
3. Wait for the response from the endpoint and put the response in a text bubble

You will not implement such an interface as it is beside the purpose of this lab.

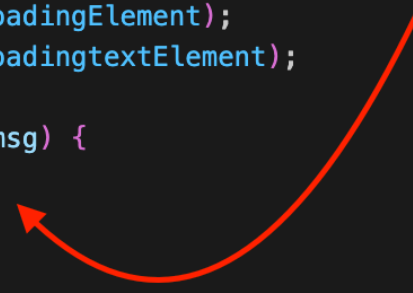
Instead, you will make sure that in step 2, you send the user input to the route you created for your chatbot earlier: <http://127.0.0.1:5000/chatbot>.

To send the input, you open `/static/script.js` and find where the endpoint is set.


```

30 const sendMessage = async (message) => {
31   // addMessage(message, 'user', 'user.jpeg');
32   addMessage(message, 'user', '../static/user.jpeg');
33   // Loading animation
34   const loadingElement = document.createElement('div');
35   const loadingtextElement = document.createElement('p');
36   loadingElement.className = `loading-animation`;
37   loadingtextElement.className = `loading-text`;
38   loadingtextElement.innerText = 'Loading....Please wait';
39   messagesContainer.appendChild(loadingElement);
40   messagesContainer.appendChild(loadingtextElement);
41
42   async function makePostRequest(msg) {
43     const url = 'www.example.com';
44     const requestBody = {
45       prompt: msg
46     };
47
48     try {
49       const response = await fetch(url, {
50         method: 'POST',
51         headers: {
52           'Content-Type': 'application/json'
53         },
54         body: JSON.stringify(requestBody)
55       });
56
57       const data = await response.text();
58       // Handle the response data here
59       console.log(data);
60       return data;
61     } catch (error) {
62       // Handle any errors that occurred during the request
63       console.error('Error:', error);
64       return error
65     }
66   }

```



Let's change this endpoint to your chatbot route. In the template, replace the default endpoint (www.example.com) with your chatbot backend URL.

<>

plaintext

'https://sinanz-5000.theianext-0-labs-prod-misc-tools-us-east-0.pro>

The URL may be different for you. Basically, copy the URL in your app launch and add `/chatbot` at the end

```
40 loadingtextElement.innerText = 'Loading...Please wait';
41 messagesContainer.appendChild/loadingElement);
42 messagesContainer.appendChild/loadingtextElement);
43
44 ✓ async function makePostRequest(msg) {
45 |   const url = 'https://efkan-5000.theia-0-labs-prod-misc-tools-us-east-0.proxy.cognitiveclass.ai/'; // Make a POST request to this url
46 |   const requestBody = {
47 |     prompt: msg
48 |   };
49
```



And that should be it! Before testing your code, let's glance at the final version of your flask app:



py



```
from flask import Flask, request, render_template
from flask_cors import CORS
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

app = Flask(__name__)
CORS(app)

model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []

@app.route('/', methods=['GET'])
def home():
    return render_template('index.html')

@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    data = request.get_json()
    print(data) # DEBUG
    input_text = data["prompt"]

    # Keep only the recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize the input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate the response
    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )
```

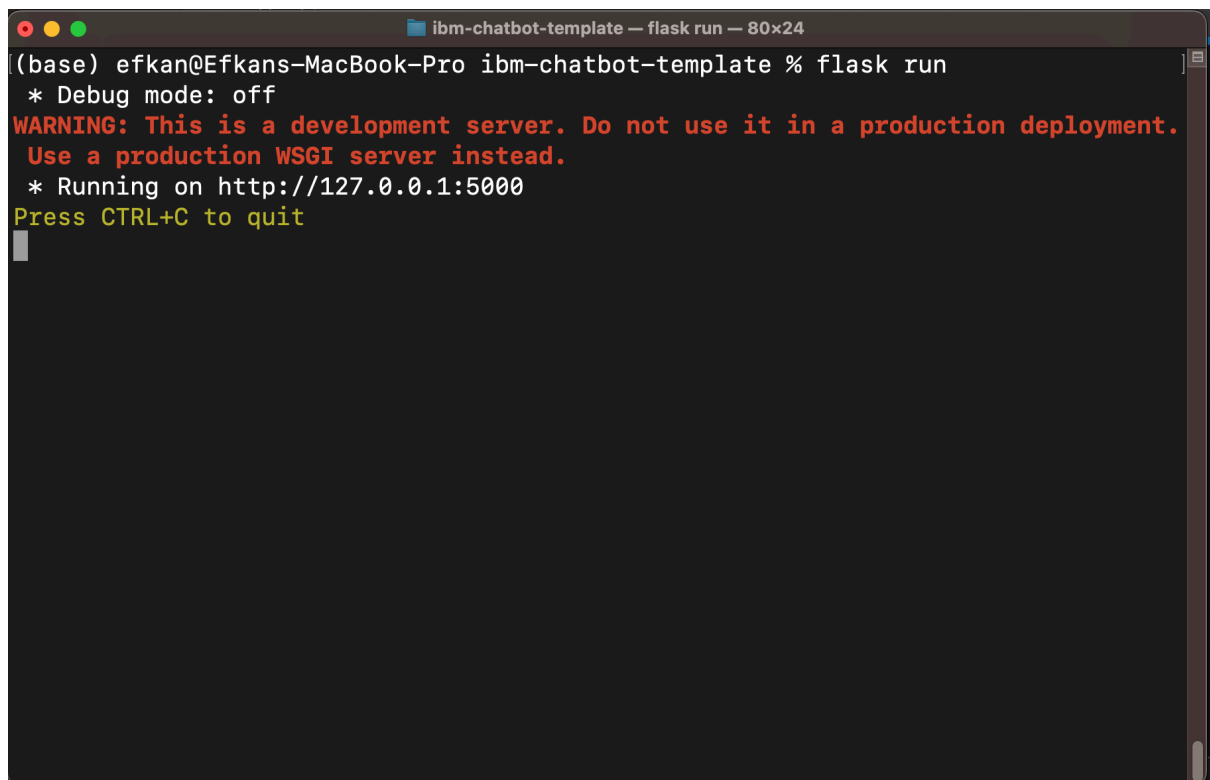
```
# Add interaction to conversation history
conversation_history.append(f"User: {input_text}")
conversation_history.append(f"Bot: {response}")

return response

if __name__ == '__main__':
    app.run()
```

Okay! Now let's test your app by stopping it first with **Ctrl + C** (if it's running) and restarting it with **flask run**.

In your terminal, you should see something like this:

A terminal window titled 'ibm-chatbot-template — flask run — 80x24' is shown. The prompt is '(base) efkan@Efkans-MacBook-Pro ibm-chatbot-template %'. The user has entered 'flask run'. The output shows: '* Debug mode: off', a red warning message 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.', '* Running on http://127.0.0.1:5000', and 'Press CTRL+C to quit'. A cursor is visible on the line following the prompt.

```
(base) efkan@Efkans-MacBook-Pro ibm-chatbot-template % flask run
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

And now let's navigate to your homepage at / and try out your chatbot!

My LLM

Copyright © JC 2023 - Created by JC with 



Hi jim, nice to meet you. My name is john. What do you do for fun?



I like hiking and skydiving. How interesting!

Hello, my name is Jim. What is your name?



I enjoy programming and kitesurfing. How about you?



Send a message...



Congratulations! If your output is similar, then you have just created your own chatbot website!

Author

Sina Nazeri (Ph.D.) [linkedin](#)

© IBM Corporation. All rights reserved.

Integrating your Chatbot into a Web Interface



Skills
Network

Introduction

In this lab, you will learn to set up a back-end server and integrate your chatbot into a web application.

Learning objectives

After completing this lab, you will be able to:

- Set up your back-end server
- Integrate your chatbot into your Flask server
- Communicate with the back-end using a web page

Prerequisites

This section assumes you know how to build the simple terminal chatbot explained in the first lab.

There are two things you must build to create your ChatGPT-like website:

1. A back-end server that hosts your chatbot
2. A front-end webpage that communicates with your back-end server

Without further ado, let's get started!

Step 1: Hosting your chatbot on a backend server

What is a backend server?

A backend server is like the brain behind a website or application. In this case, the backend server will receive prompts from your website, feed them into your chatbot, and return the output of the chatbot back to the website, which will be read by the user.

Hosting a simple backend server using Flask

flask is a Python framework for building web applications with Python. It provides a set of tools and functionalities to handle incoming requests, process data, and generate responses, making it easy to power your website or application.

Prerequisites

For all terminal interactions in this lab (such as running python files or installing packages), you will use the built-in terminal that comes with Cloud IDE. You may launch the terminal by either:

- Pressing **Ctrl + `** , or
- By selecting Terminal --> New Terminal from the toolbar at the top of the IDE window on the right.

In your terminal, let's install the following requisites:



A terminal window with a light gray background. The top bar contains a code icon on the left, a 'bash' label in a purple pill, and a copy icon on the right. The main area contains two lines of text: 'python3.11 -m pip install flask' and 'python3.11 -m pip install flask_cors'. On the right side of the main area is a green 'Run' button with a play icon.

```
python3.11 -m pip install flask
python3.11 -m pip install flask_cors
```

Setting up the server

Next, you will create a script that stores your flask server code.

To create a new Python file, Click on **File Explorer** , then right-click in the explorer area and select **New File** . Name this new file **app.py** .

Let's take a look at how to implement a simple flask server:



A code editor window with a light gray background. The top bar contains a code icon on the left, a 'py' label in a purple pill, and a copy icon on the right. The main area contains Python code for a simple flask server.

```
from flask import Flask

app = Flask(__name__)

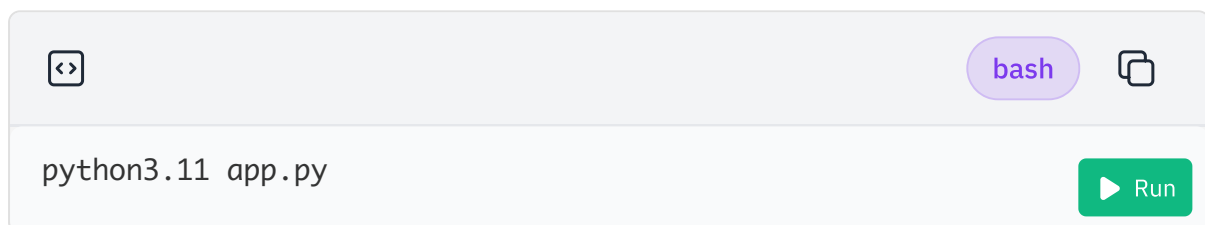
@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
```

Paste the above code in the `app.py` file you just created and save it.

In this code:

- You import the `Flask` class from the `flask` module.
- You create an instance of the `Flask` class and assign it to the variable `app`.
- You define a route for the homepage by decorating the `home()` function with the `@app.route()` decorator. The function returns the string 'Hello, World!'. This means that when the user visits the URL where the website is hosted, the backend server will receive the request and return 'Hello, World!' to the user.
- The `if __name__ == '__main__':` condition ensures that the server is only run if the script is executed directly, not when imported as a module.
- Finally, you call `app.run()` to start the server.



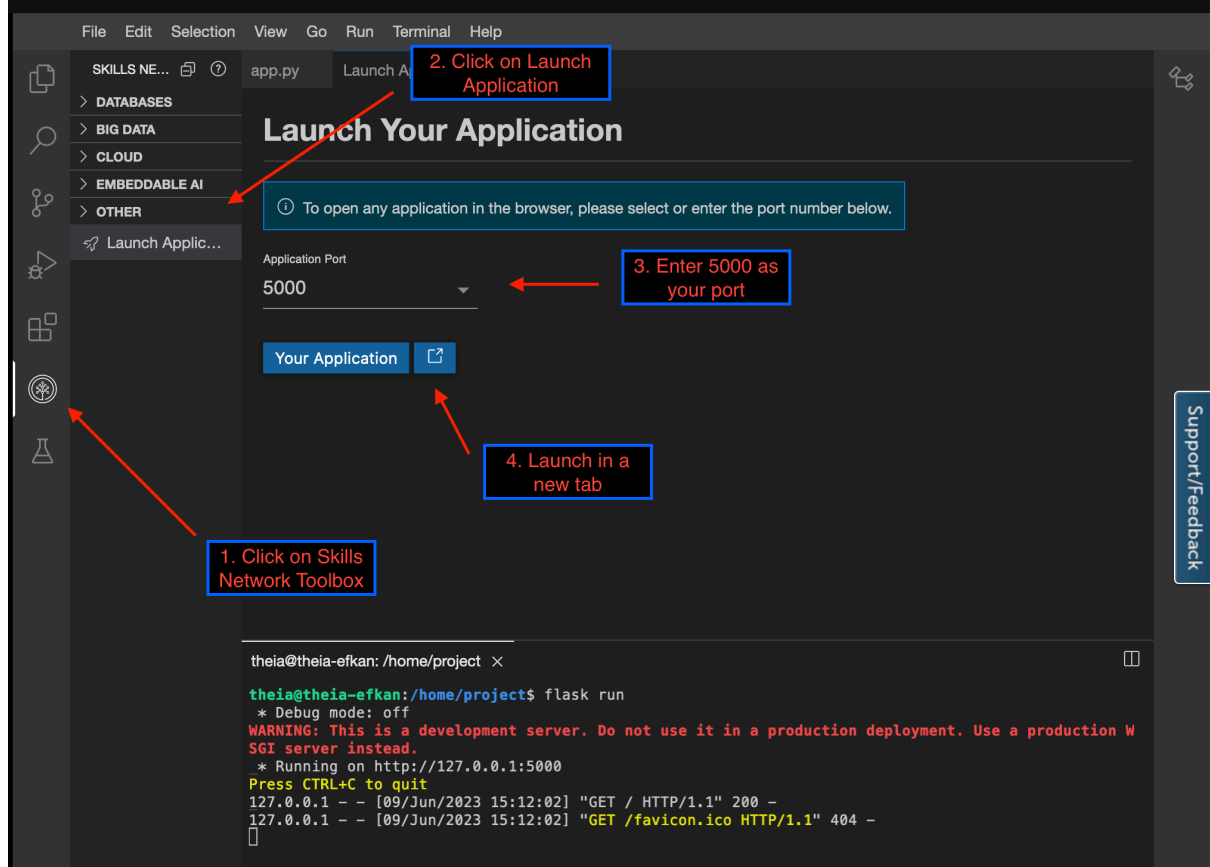
A terminal window with a light gray background. At the top left is a code icon (two arrows pointing in opposite directions). At the top right is a 'bash' label in a purple pill and a copy icon. The main area contains the text 'python3.11 app.py'. At the bottom right is a green 'Run' button with a play icon.

Save this code in a Python file, for example, `app.py`, and run it by typing `python app.py` in the terminal. By default, Flask hosts the server at <http://127.0.0.1:5000/> (which is equivalent to <http://127.0.0.1:5000/>).

With this command, the Flask server will start running. If you run this server on your local machine, then you can access it by visiting <http://127.0.0.1:5000/> or <http://localhost:5000/> in your web browser.

However, you are currently running this lab in the Skills Network Cloud. Thus, you can access your server as follows:

1. Navigate to the Skills Network Toolbox from the toolbar on the left side of the IDE
2. Click "Launch Application" in the adjacent vertical sidebar
3. Enter 5000 as your Application Port
4. Launch the application in a new browser tab

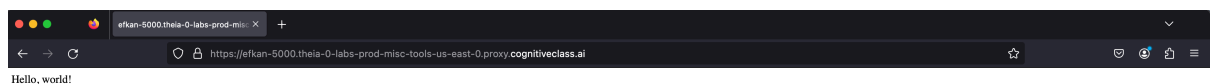


By performing the above steps, you have visited the **relative localhost URL** of the cloud server.

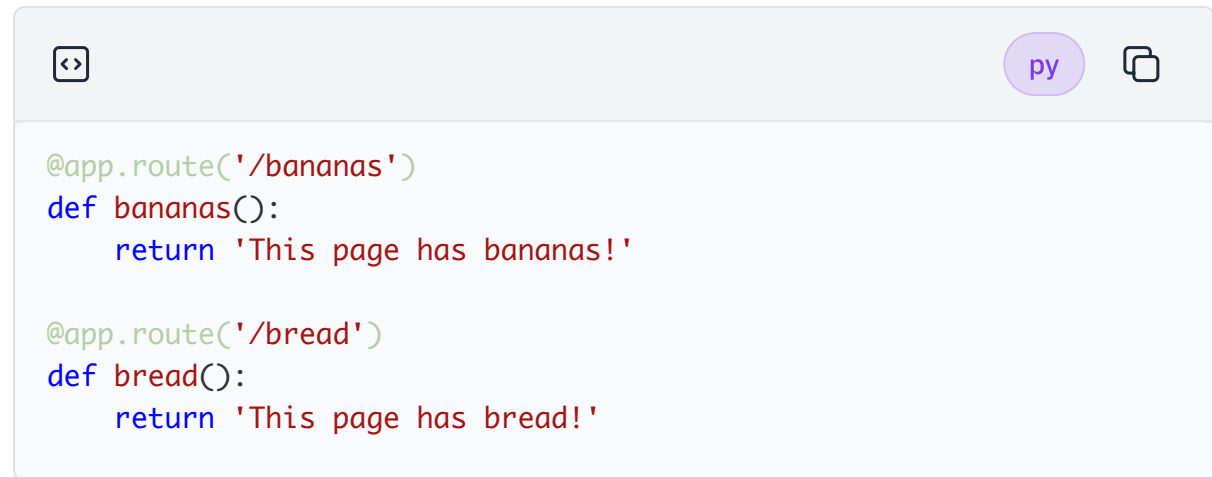
IMPORTANT: Throughout the rest of this lab, you will refer to this URL as .

On visiting the localhost, you should see the "Hello, World!" message displayed.

Here's what it should look like:



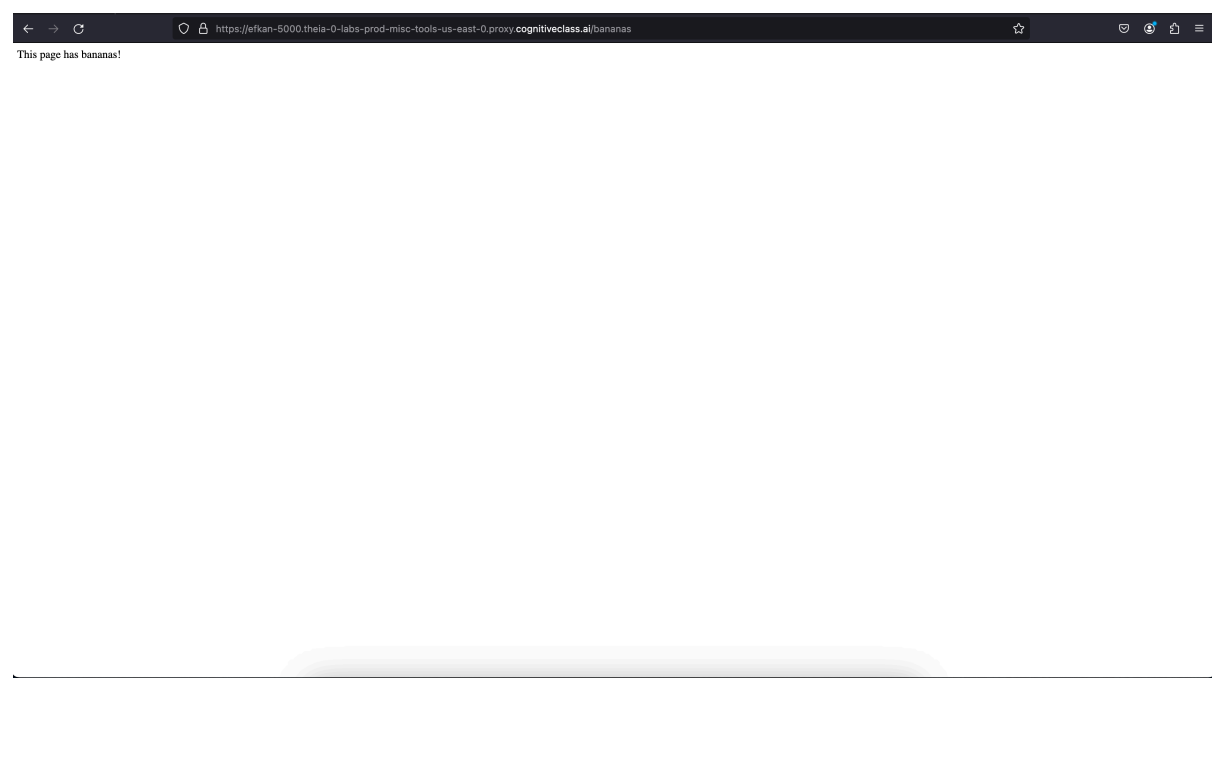
Let's add the following routes to try it out:

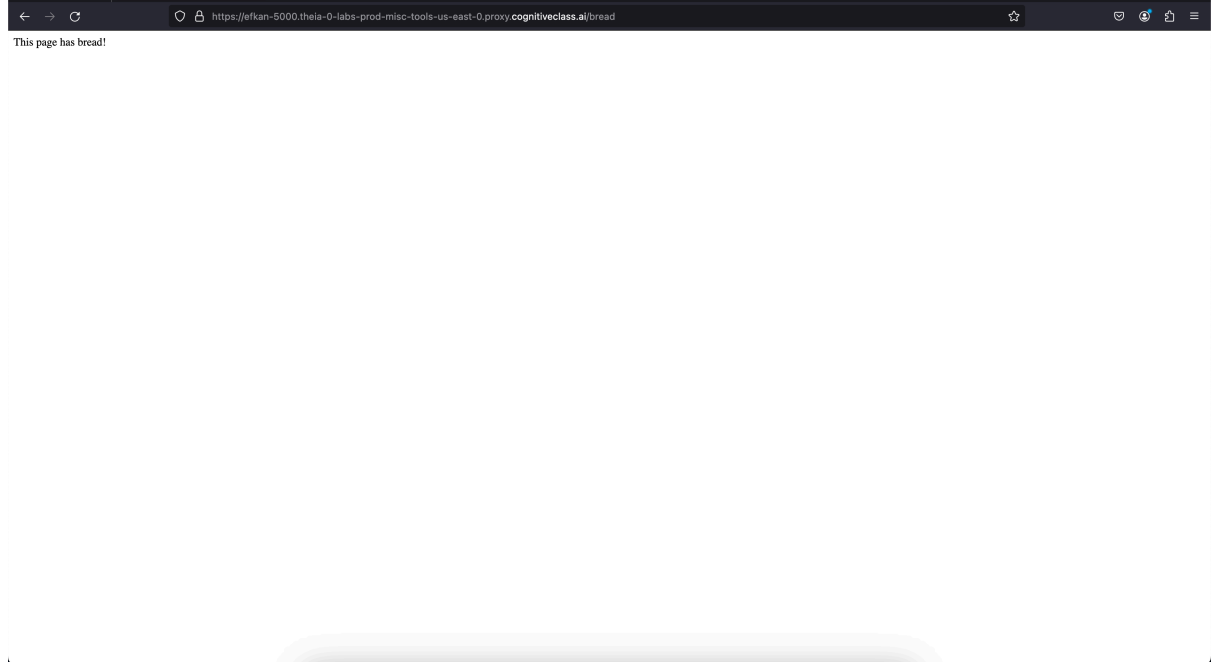


```
@app.route('/bananas')
def bananas():
    return 'This page has bananas!'

@app.route('/bread')
def bread():
    return 'This page has bread!'
```

Now, let's stop our app using **Ctrl + C** in the terminal and re-run with **flask run**. Then, let's visit both of these routes at /bananas and /bread. Here's what you should see:





Okay - now that you've demonstrated how routes work, you can remove these two routes (bananas and bread) from your `app.py` as you won't be using them.

Before proceeding, you'll also add two more lines of code to your program to mitigate CORS errors - a type of error related to making requests to domains other than the one that hosts this webpage. You'll be modifying your code as follows:

```
from flask import Flask
from flask_cors import CORS      # newly added

app = Flask(__name__)
CORS(app)                        # newly added

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

Integrating your chatbot into your Flask server

Now that you have your Flask server set up, let's integrate your chatbot into your Flask server.

As stated at the beginning, this lab assumes you've completed the first lab of this

First, you'll install the requisites We pin library versions to ensure the code runs consistently without breaking due to future updates or changes in dependencies.

plaintext

```
python3.11 -m pip install transformers==4.41.2
python3.11 -m pip install torch==2.2.2
python3.11 -m pip install accelerate==0.30.1
python3.11 -m pip install numpy==1.26.4
```

Next, let's copy the code to initialize your chatbot from lab 1 and place it at the top of your script. You also must import the necessary libraries for your chatbot.

py

```
from transformers import AutoModelForSeq2SeqLM
from transformers import AutoTokenizer
```

py

```
model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []
```

Next, you'll need to import a module to read the data.

py

```
from flask import request
```

Before implementing the actual function though, you need to determine the structure you expect to receive in the incoming HTTP request.

Let's define your expected structure as follows:

 json 

```
{  
  "prompt": "message"  
}
```

Now implement your chatbot function. Again, you'll copy code over from your chatbot implementation from the first lab.



py



```
@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    data = request.get_json()
    input_text = data["prompt"]

    # Keep only recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate response
    outputs = model.generate(
        *inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    # Decode response
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Save conversation
    conversation_history.append(f"User: {input_text}")
    conversation_history.append(f"Bot: {response}")

    return response
```

The only new thing you've done up till now is read the prompt from the HTTP request body. You've copied everything else from your previous chatbot implementation!



py



```
from flask import Flask, request, render_template
from flask_cors import CORS
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

app = Flask(__name__)
CORS(app)

model_name = "facebook/blenderbot-400M-distill"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
conversation_history = []

@app.route('/chatbot', methods=['POST'])
def handle_prompt():
    # Read prompt from HTTP request body
    data = request.get_json()
    input_text = data["prompt"]

    # Keep only the recent conversation
    conversation_history[:] = conversation_history[-6:]

    # Create conversation history string
    history = "\n".join(conversation_history)
    prompt = history + f"\nUser: {input_text}\nBot:"

    # Tokenize the input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    # Generate the response
    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    # Decode the response
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)

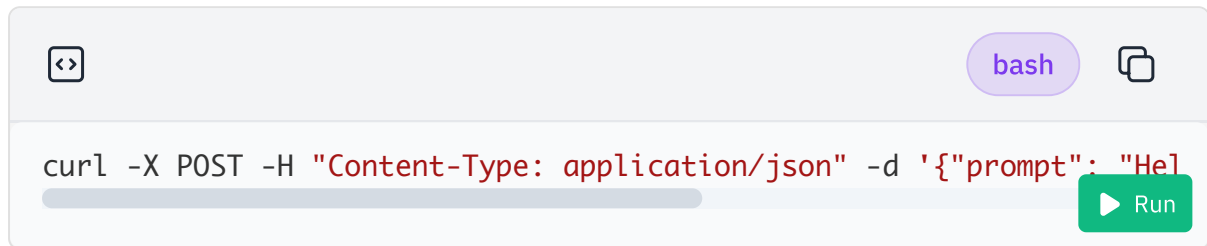
    # Add interaction to conversation history
```

```
    return response

if __name__ == '__main__':
    app.run()
```

Now let's test your implementation by using `curl` to make a POST request to /chatbot with the following request body: {'prompt': 'Hello, how are you today?'}.

Open a new terminal: Select terminal tab --> open new terminal

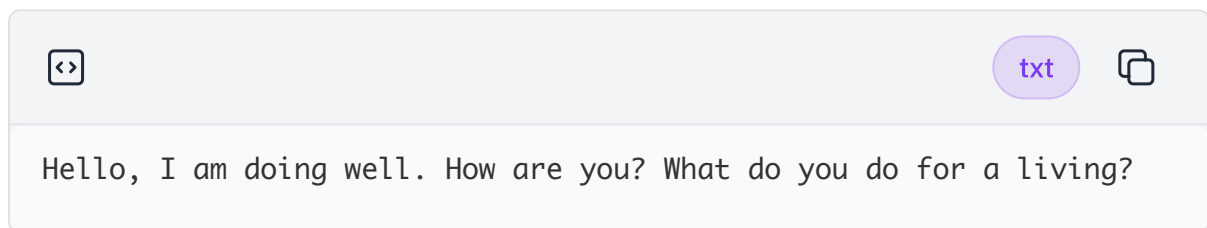
A terminal window with a light gray background. At the top, there is a toolbar with a code icon (two arrows pointing left and right), a purple button labeled 'bash', and a copy icon (two overlapping squares). The main area of the terminal contains the command: `curl -X POST -H "Content-Type: application/json" -d '{"prompt": "Hel`. A green button with a white play icon and the text 'Run' is positioned at the bottom right of the command line.

```
<> bash
```

```
curl -X POST -H "Content-Type: application/json" -d '{"prompt": "Hel
```

```
▶ Run
```

Here's the output of the above code:

A terminal window with a light gray background. At the top, there is a toolbar with a code icon (two arrows pointing left and right), a purple button labeled 'txt', and a copy icon (two overlapping squares). The main area of the terminal contains the response text: 'Hello, I am doing well. How are you? What do you do for a living?'.

```
<> txt
```

```
Hello, I am doing well. How are you? What do you do for a living?
```

If you got a similar response, then congratulations! You have successfully created a Flask backend server with an integrated chatbot!

After you finish, press `Ctrl + c` to stop the server.

Communicating with your... →